TERRAFORM TASK -05

1.Execute the script shown in the video.

Terraform Remote state and state locking: We can store terraform configuration files and state file in github or any other repository but it is not good practise.

We use s3,terraform storage,hashicorp consul to store the state file.

State locking is used to lock the state file so that no two users can execute the state file at the same point of time.

Remote backend and state locking: We can use s3 as remote backend and dynamo db for state locking.

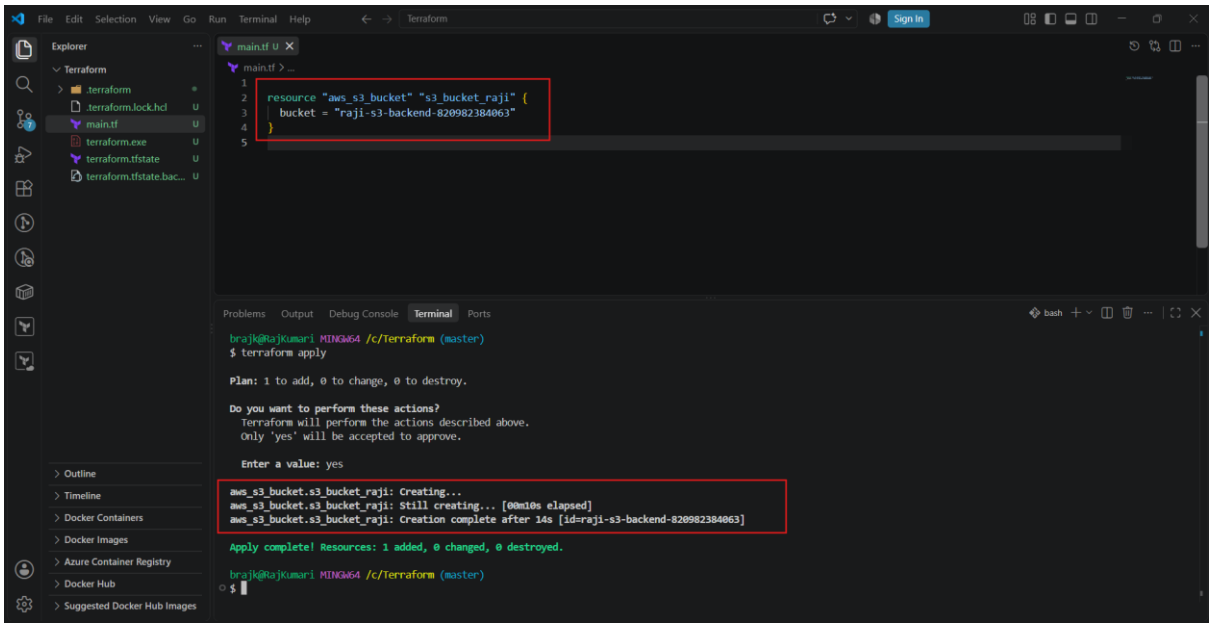
Create s3 using terraform:

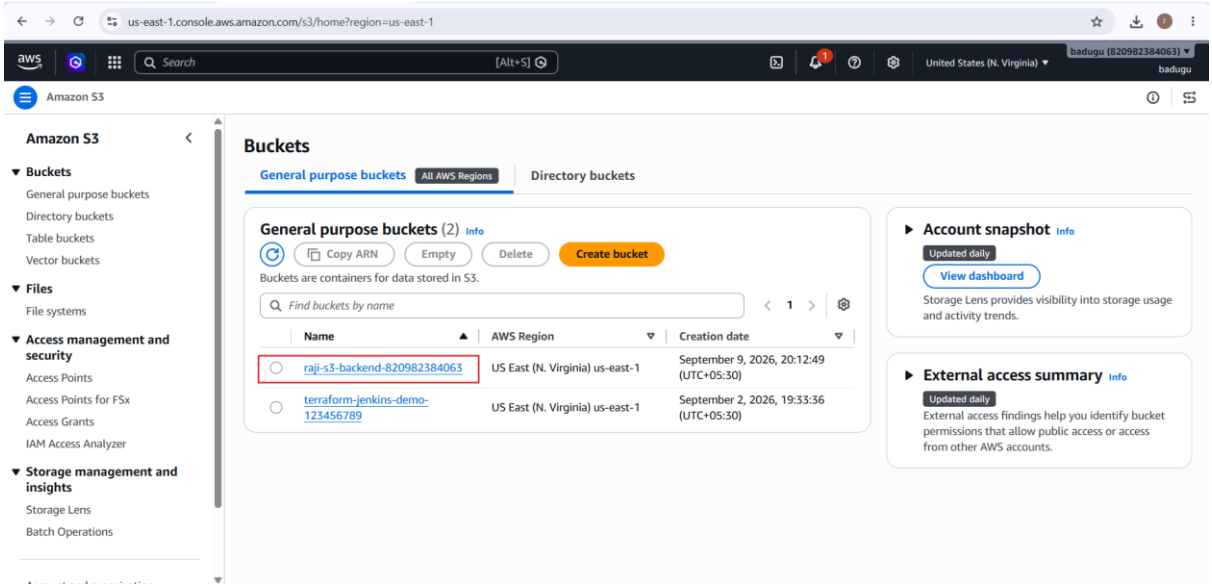
```
resource "aws_s3_bucket" "s3_bucket" {

  bucket = "s3backend"

  acl = "private"

}
```



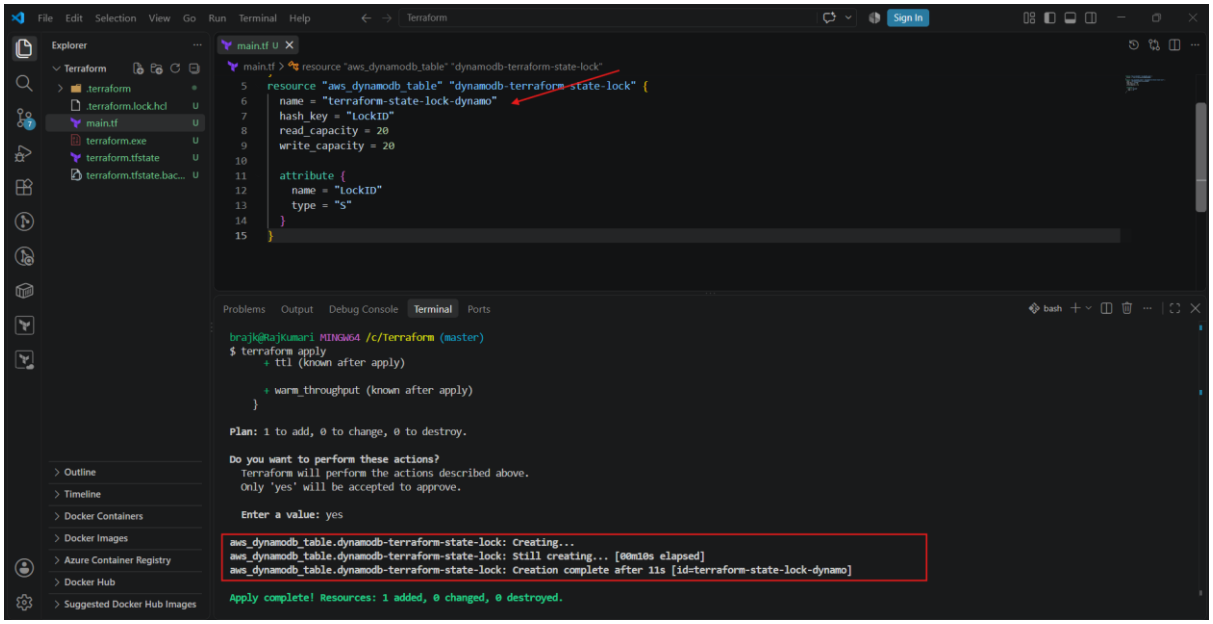


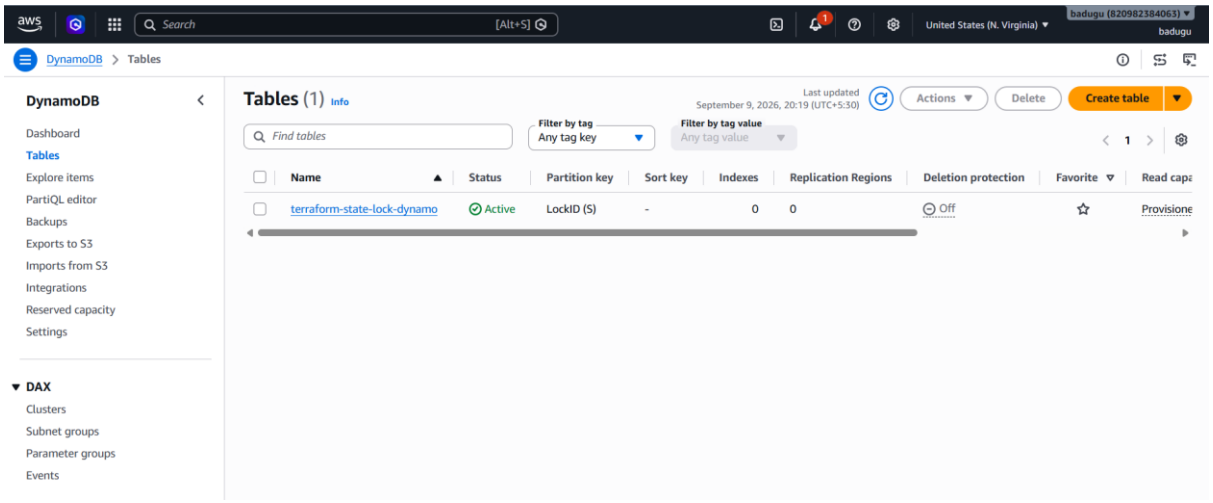
Create dynamo db using terraform:

==========

```
resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {
  name = "terraform-state-lock-dynamo"
  hash_key = "LockID"
  read_capacity = 20
  write_capacity = 20

  attribute {
    name = "LockID"
    type = "S"
  }
}
```

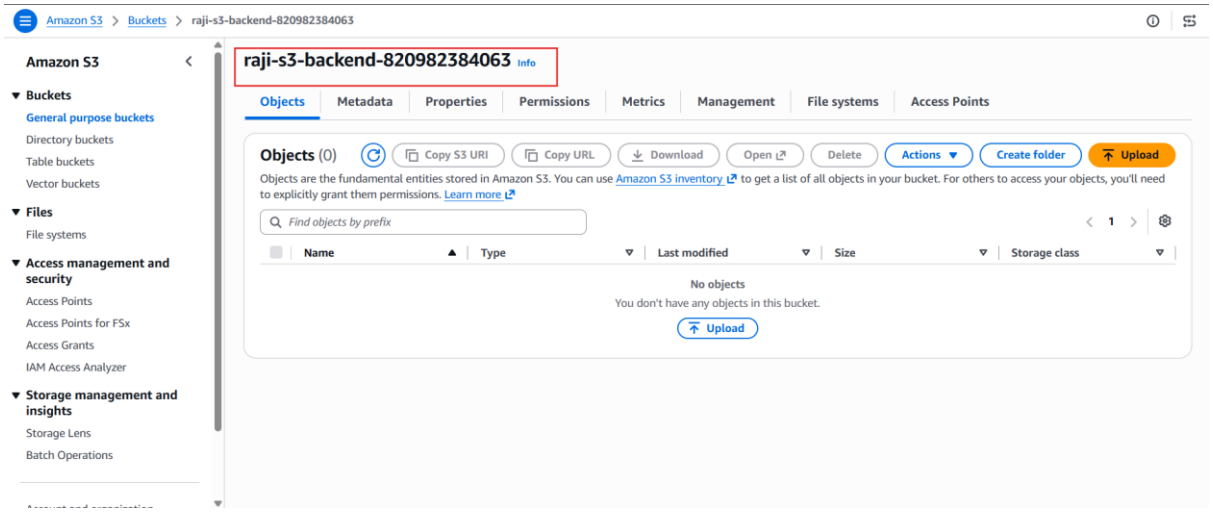


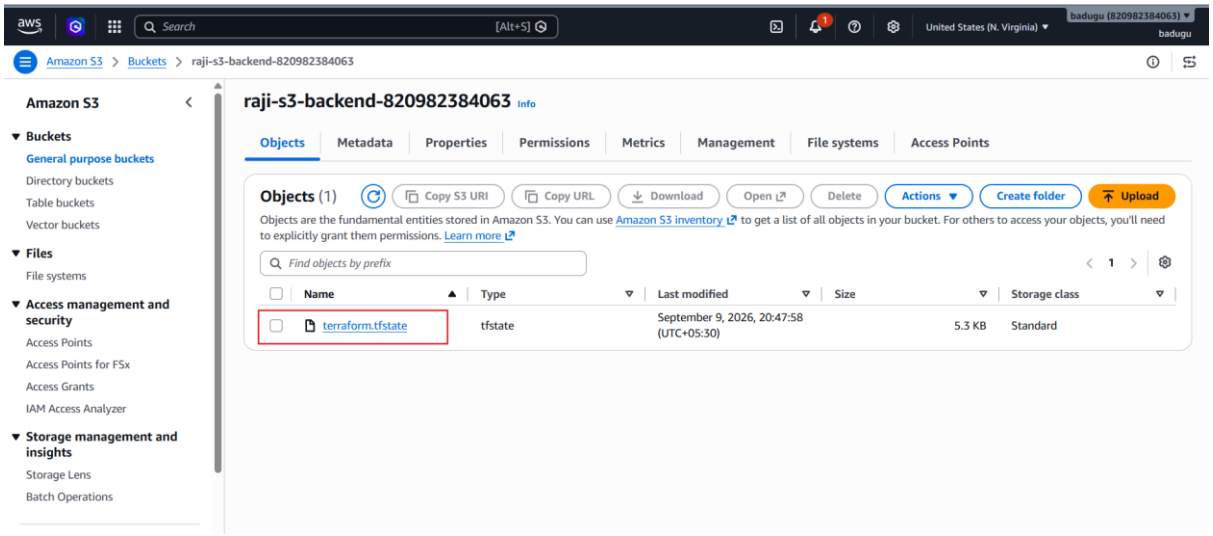


S3 as backend for terraform.tfstate file:

==========

```
terraform {
  backend "s3" {
    bucket = "s3backend"
    dynamodb_table = "terraform-state-lock-dynamo"
    key    = "terraform.tfstate"
    region = "us-east-1"
  }
}
```





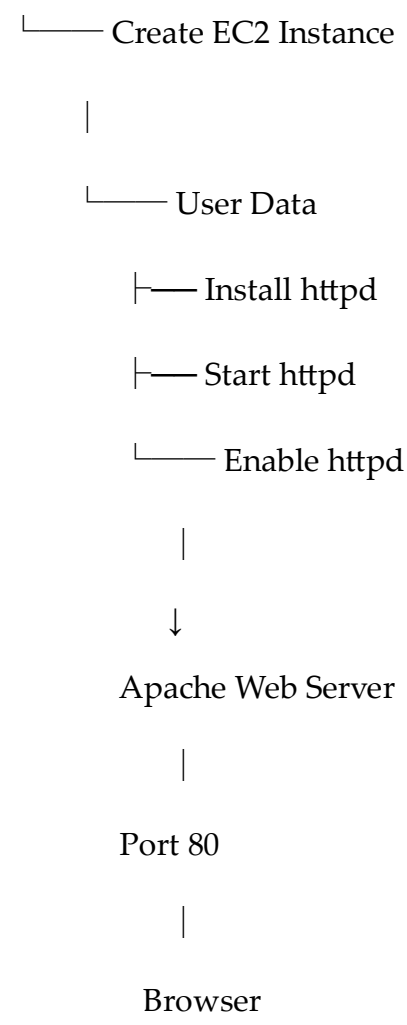
2.Create one **EC2 instance** with **httpd installed** using a Terraform script

Terraform will create **one EC2 instance**, install **Apache HTTP Server (httpd)**, start it, and allow HTTP traffic so you can open the web page in your browser.

Architecture

Terraform

```
|
├── Create Security Group
|       └── Allow SSH : 22
|       └── Allow HTTP : 80
|
```



Step 1: Create a Terraform folder

On your Windows PC, create a folder:

ec2-httpd-terraform

Open this folder in VS Code.

You can create these files:

ec2-httpd-terraform/

├── provider.tf

├── main.tf

├── variables.tf

└── outputs.tf

For a beginner, we can also do everything in main.tf, but separating the files is better for learning.

Step 2: Create provider.tf

Put this in provider.tf:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}


provider "aws" {
  region = "us-east-1"
}
```

Explanation

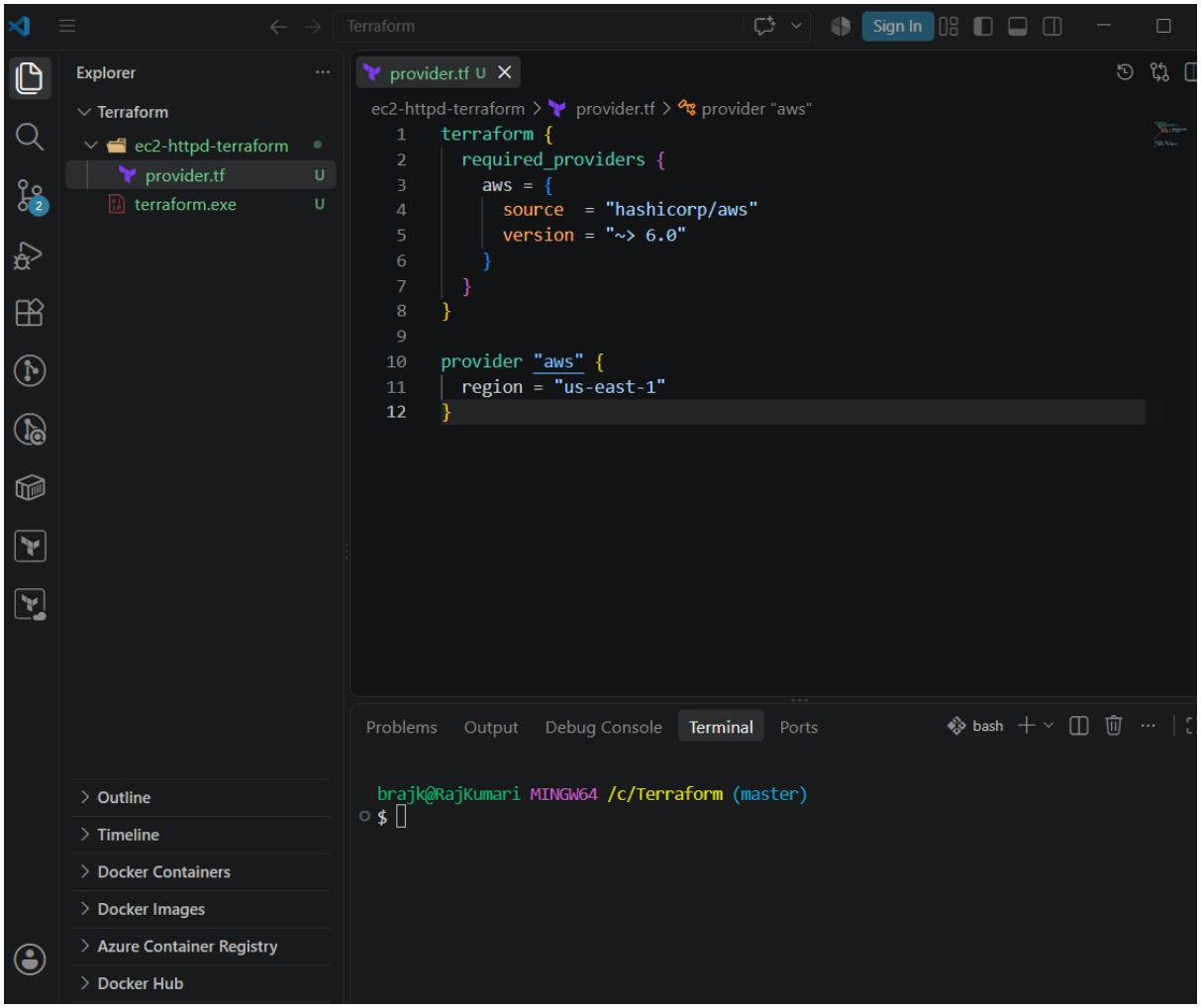
provider "aws"

tells Terraform that we are using AWS.

region = "us-east-1"

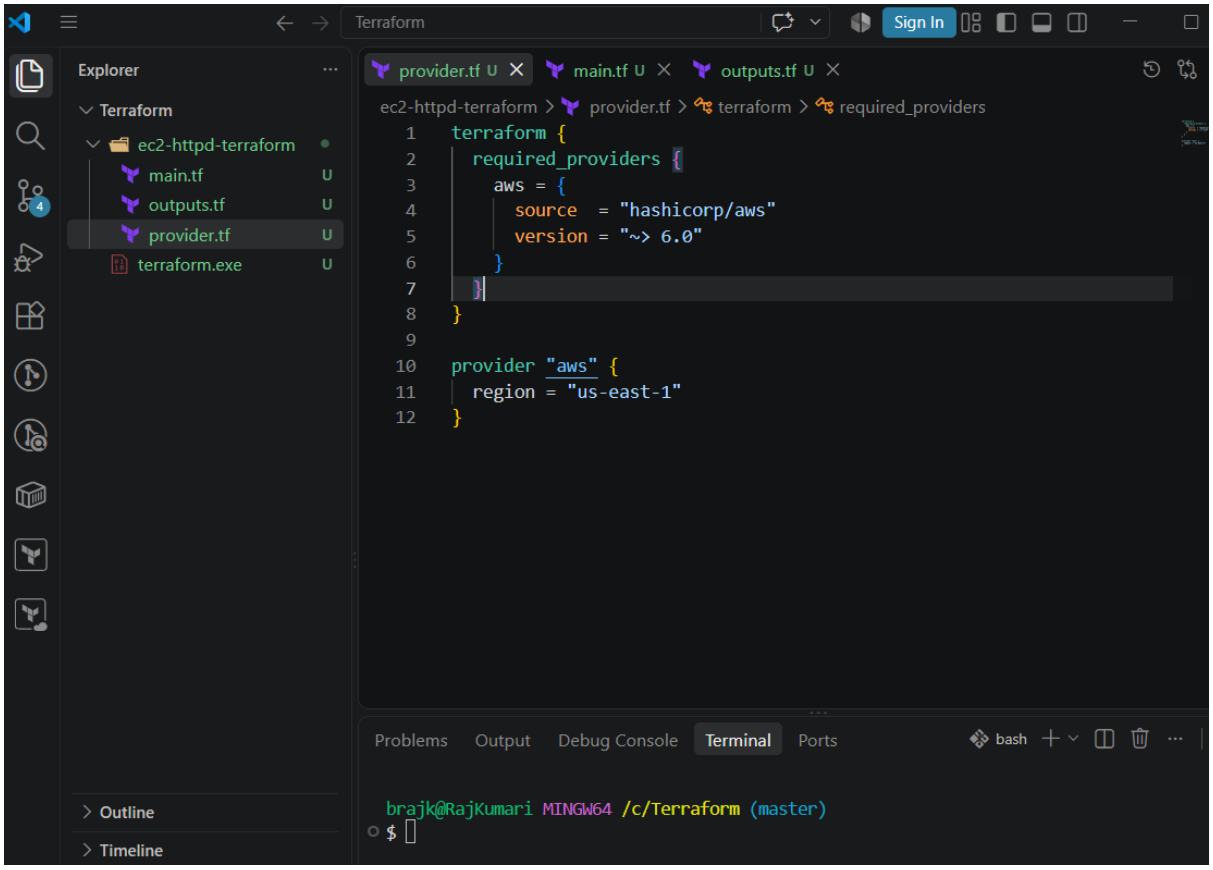
means Terraform will create the resources in **US East (N. Virginia)**.

You can change this if your AWS resources are in another region.



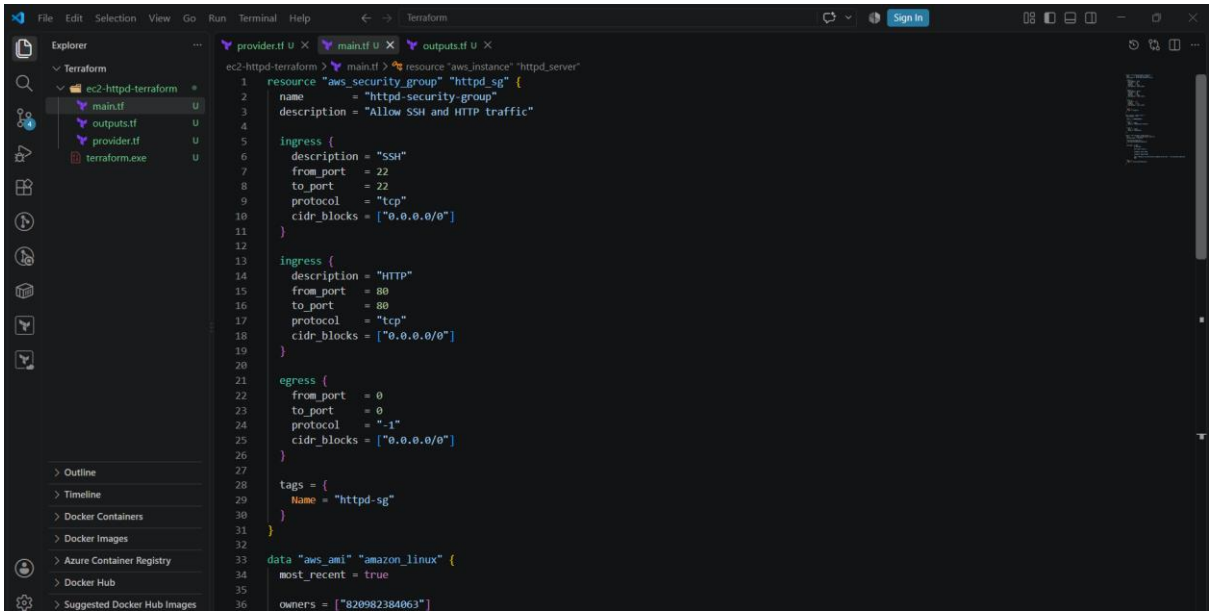
Step 2: Create provider.tf

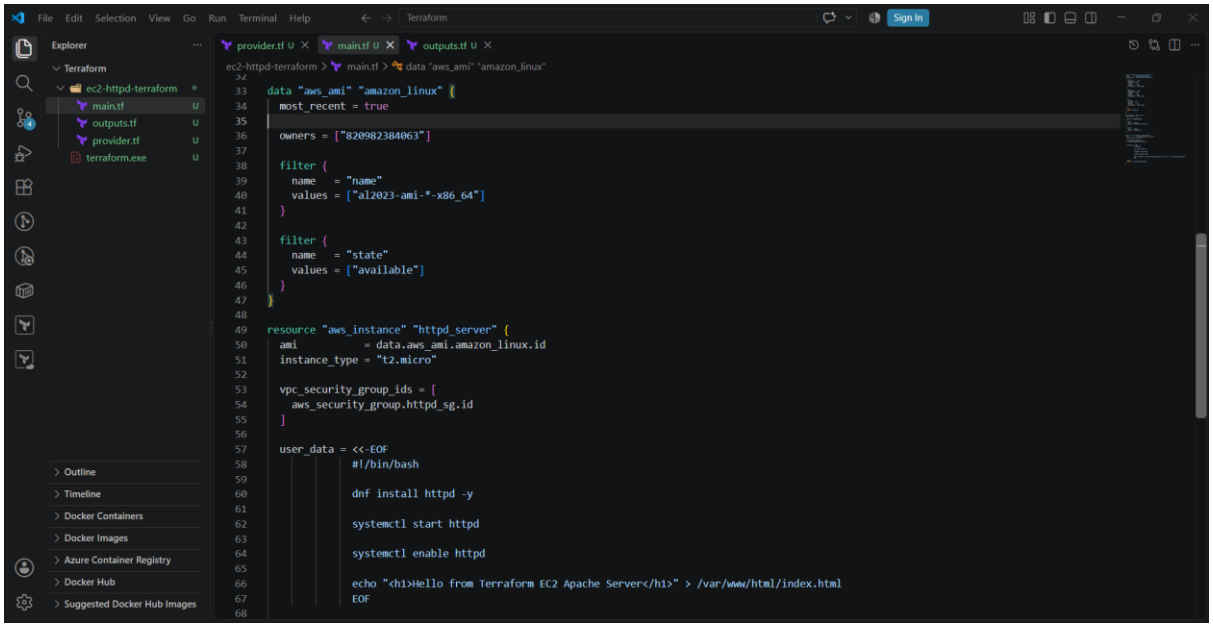
Put this in provider.tf:



Step 3: Create main.tf

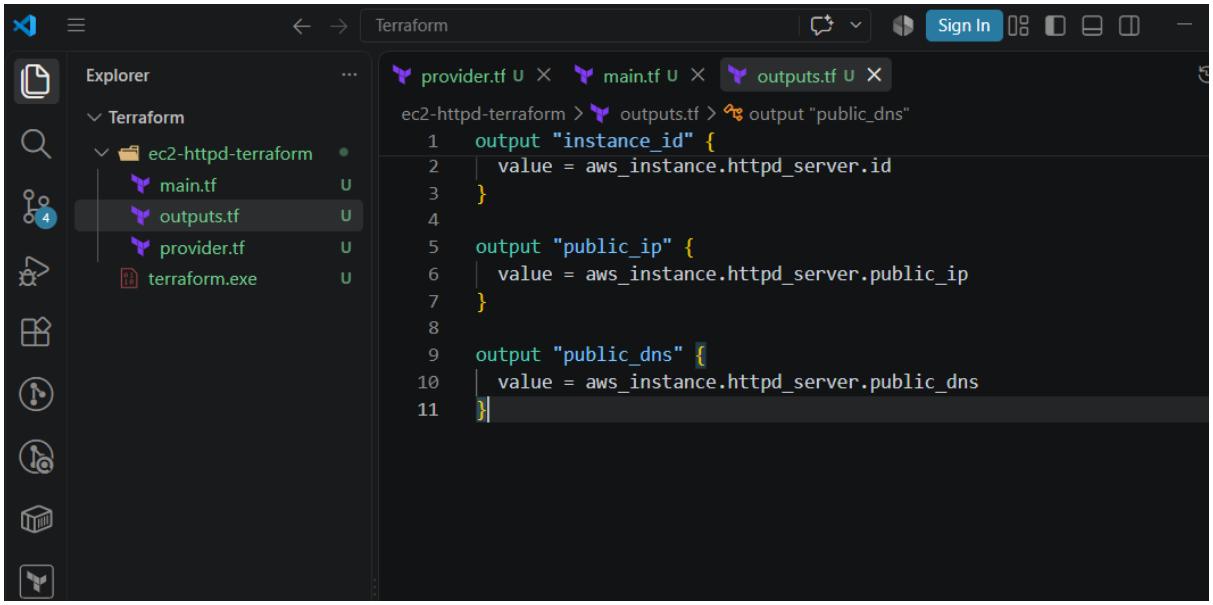
Put this in main.tf:





Step 5: Create outputs.tf

Put this in outputs.tf:



Step 6: Configure AWS credentials

Before Terraform can create the EC2 instance, your AWS CLI must be configured with the correct account.

Check:

aws sts get-caller-identity

Make sure the account shown is your **new AWS account**.

Also check:

aws configure list

You should see your configured region and credentials.

Step 7: Initialize Terraform

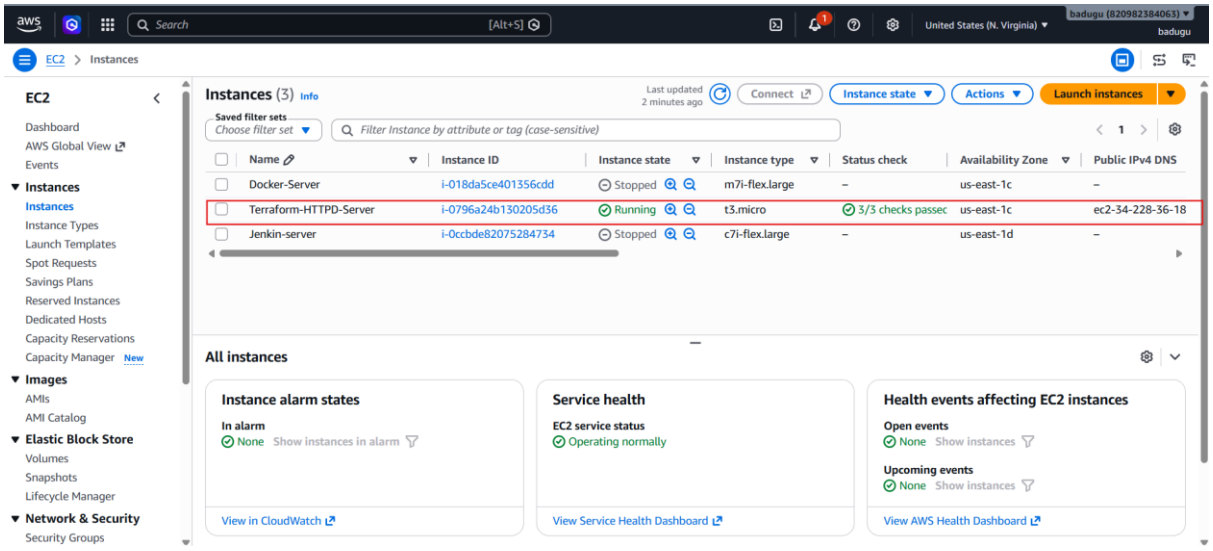
Open the terminal inside your Terraform folder.

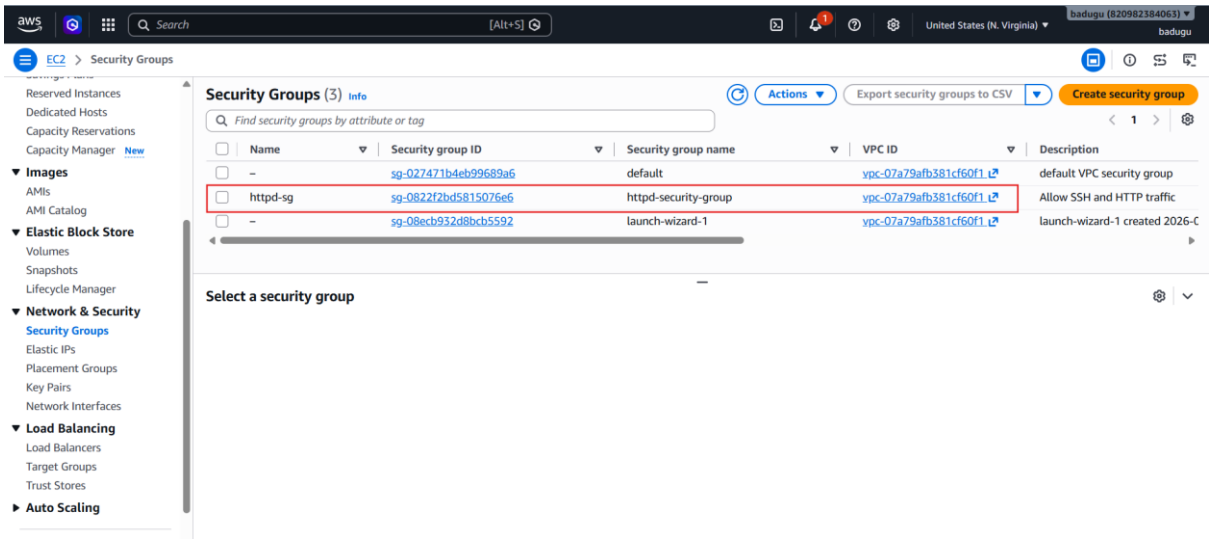
Run:

terraform init

You should see something similar to:

Initializing the backend...





3.Set up **S3 as backend** for task 3.

For **Task 3**, you can configure **Amazon S3 as the Terraform remote backend** so that terraform.tfstate is stored in S3 instead of only on your local computer.

Since you already created an S3 bucket in your new AWS account, let's do it step by step.

1. What are we going to configure?

Normally Terraform stores state locally:

Terraform

↓

terraform.tfstate

↓

Your local PC

With an S3 backend:

AWS

|

Terraform ———┤

↓

S3 Bucket

↓

terraform.tfstate

This is useful because the state can be shared by a team and is not dependent on one developer's laptop.

Step 1: Make sure you're using the new AWS account

You already switched your CLI, but verify:

aws sts get-caller-identity

You should see:

Account: 820982384063

Step 2: Check your existing S3 bucket

You previously showed that this bucket exists:

terraform-jenkins-demo-123456789

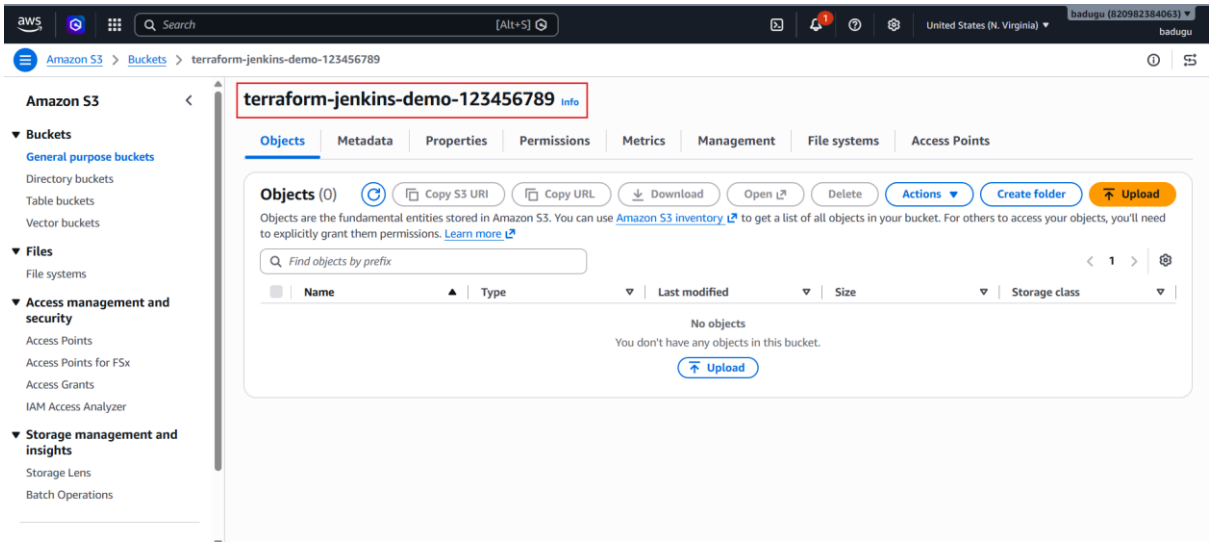
Check it:

aws s3 ls

You should see:

2026-09-02 19:33:36 terraform-jenkins-demo-123456789

We can use this bucket as the Terraform backend.



Step 3: Add the S3 backend to Terraform

In your Terraform project, open provider.tf.

Understand these three values

bucket = "terraform-jenkins-demo-123456789"

This is the S3 bucket where Terraform stores the state.

key = "task3/ec2-httpd/terraform.tfstate"

This is the **path/name of the state file inside the bucket**.

Your S3 bucket will look conceptually like:

terraform-jenkins-demo-123456789

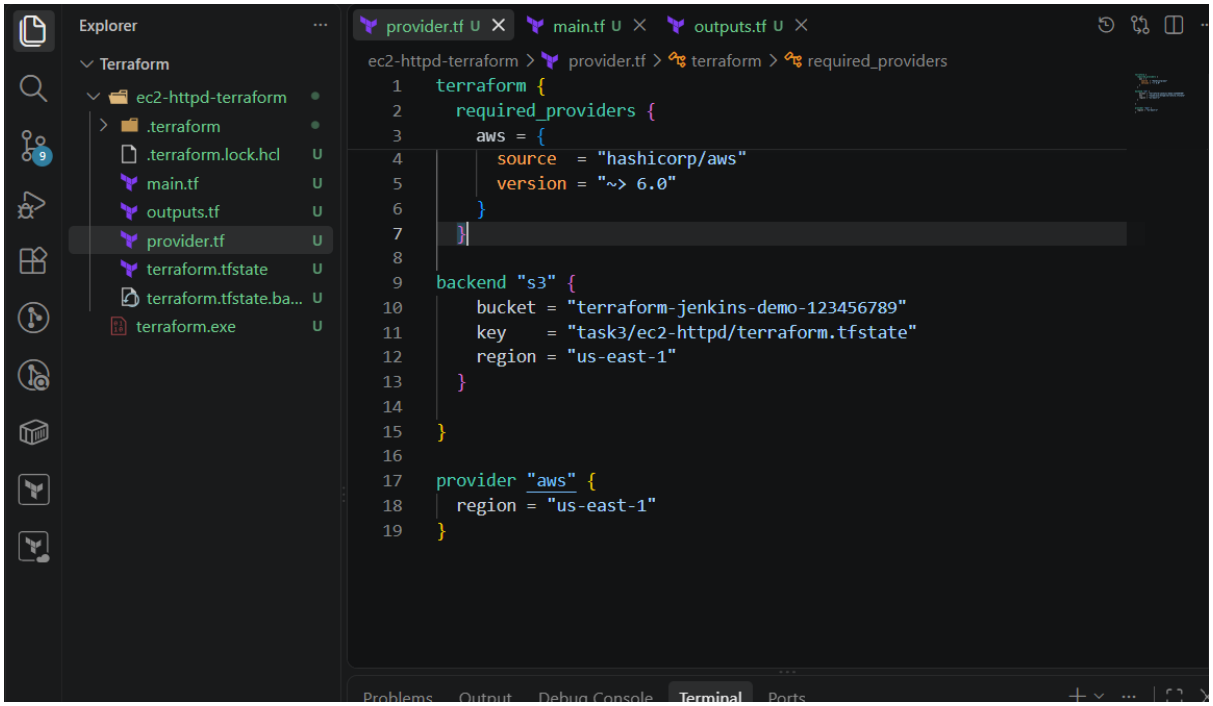
└── task3/

└── ec2-httpd/

└──── terraform.tfstate

region = "us-east-1"

This is the region of the S3 bucket.



Step 4: Save the file

Your project should now look like:

ec2-httpd-terraform/

│

├── provider.tf

├── main.tf

├── outputs.tf

└──── ...

Step 5: Initialize the S3 backend

Run:

terraform init

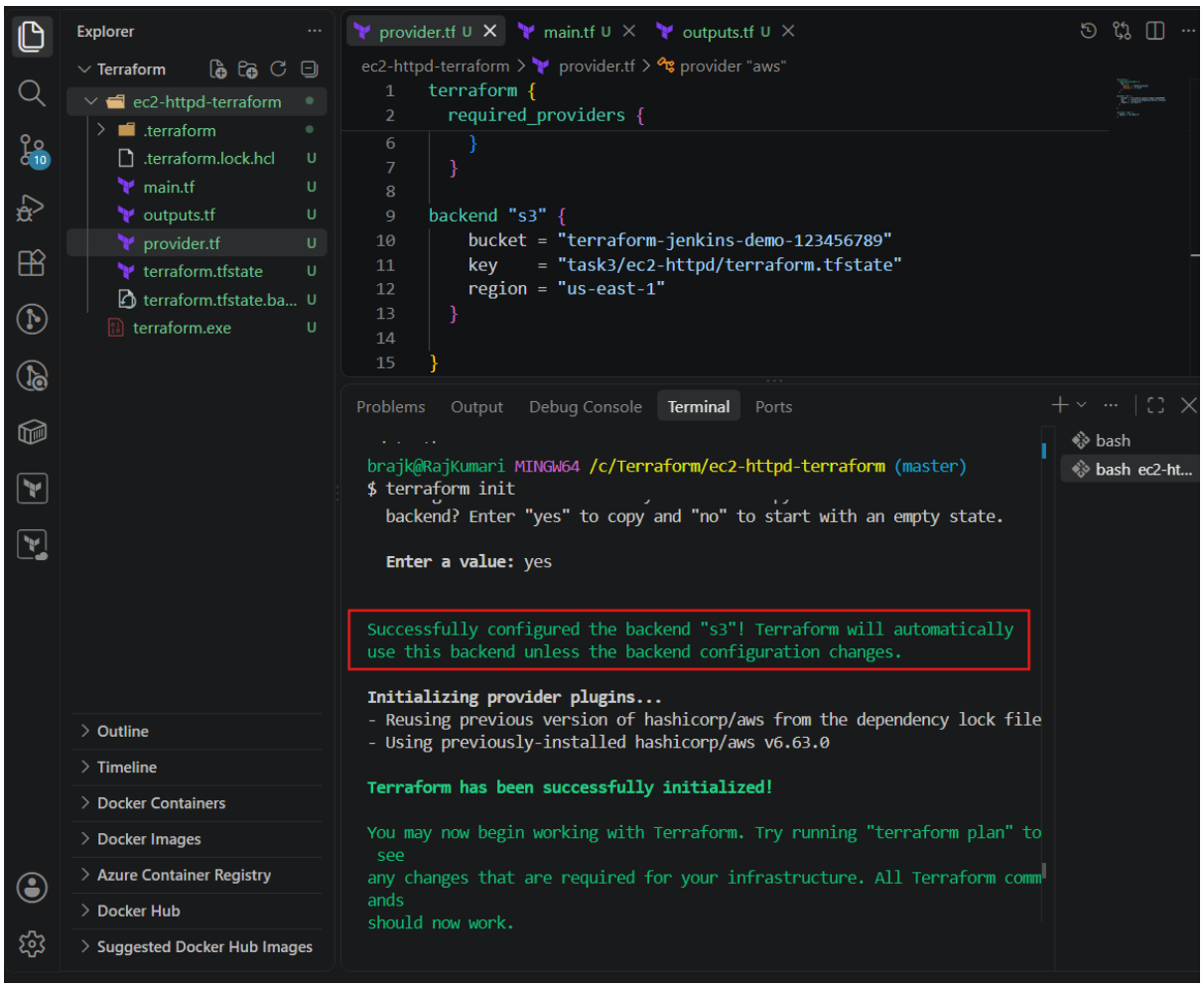
Because you changed the backend configuration, Terraform will detect the new backend.

You may see:

Initializing the backend...

and eventually:

Successfully configured the backend "s3"!



Step 6: If Terraform asks to migrate the state

Because you already ran terraform apply, you probably have a local:

terraform.tfstate

Terraform may ask:

Do you want to copy existing state to the new backend?

Choose:

yes

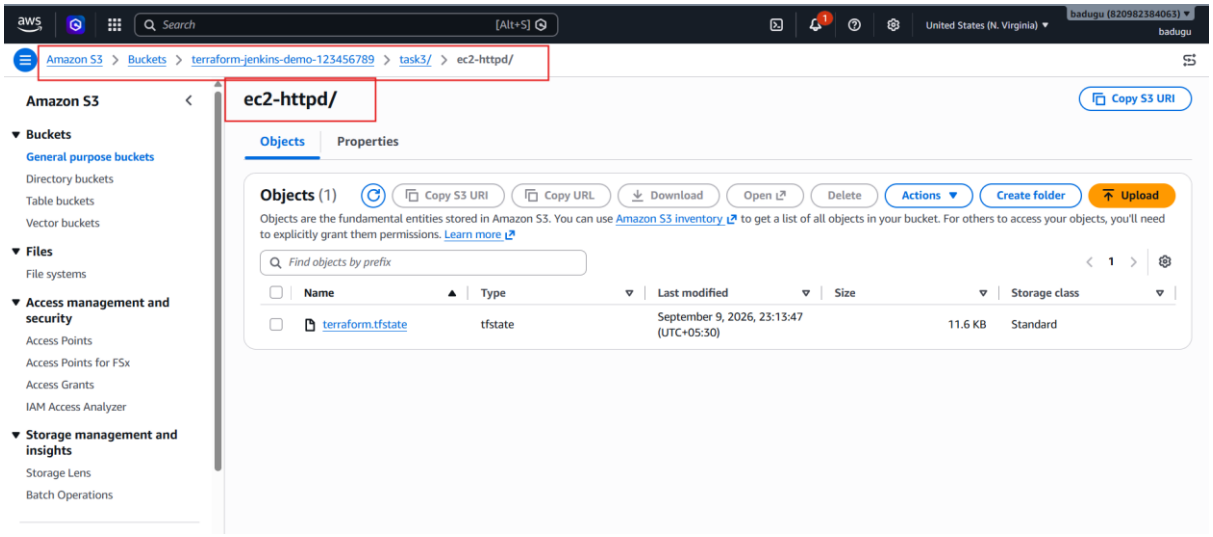
This is important.

Terraform will take your existing local state and upload it to:

S3

↓

task3/ec2-httpd/terraform.tfstate



Set up **DynamoDB locking** for task 3.

1. What is DynamoDB locking?

Without locking:

Developer A ──┐

├──> terraform.tfstate

Developer B ──┘

Both could run terraform apply at the same time, potentially causing state conflicts.

With locking:

Developer A

|

↓

DynamoDB Lock

|

↓

Terraform State in S3

|

↓

Developer B → WAIT

DynamoDB acts as a **lock mechanism**, while S3 stores the actual terraform.tfstate.

Step 1: Create the DynamoDB table

Use AWS CLI:

```
aws dynamodb create-table \
  --table-name terraform-state-lock \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST \
  --region us-east-1
```

You should get a response showing:

TableName: terraform-state-lock

TableStatus: CREATING

Wait a few seconds.

Step 2: Verify the table

Run:

```
aws dynamodb describe-table \
  --table-name terraform-state-lock \
```

--region us-east-1 \

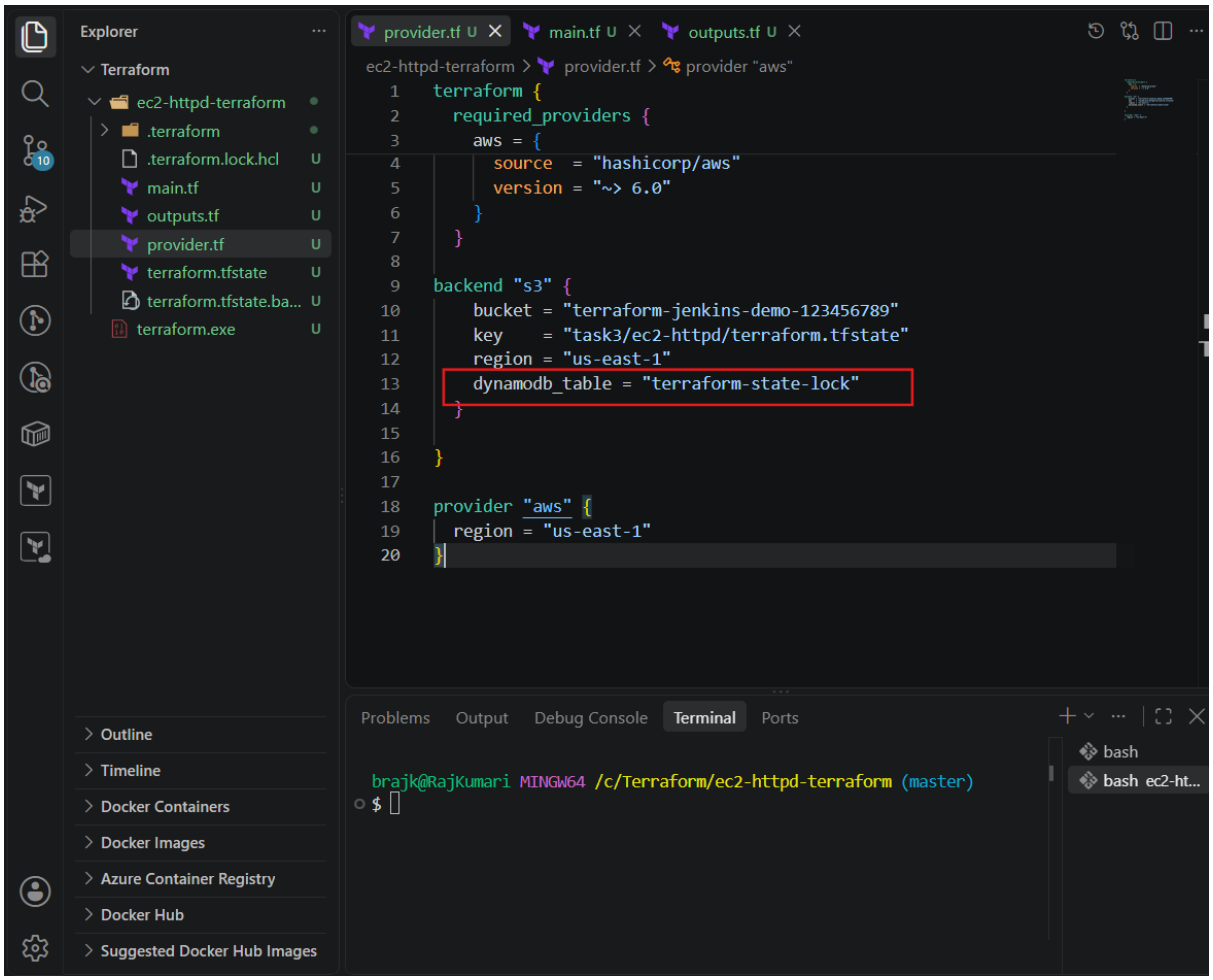
--query "Table.{Name:TableName,Status:TableStatus}"

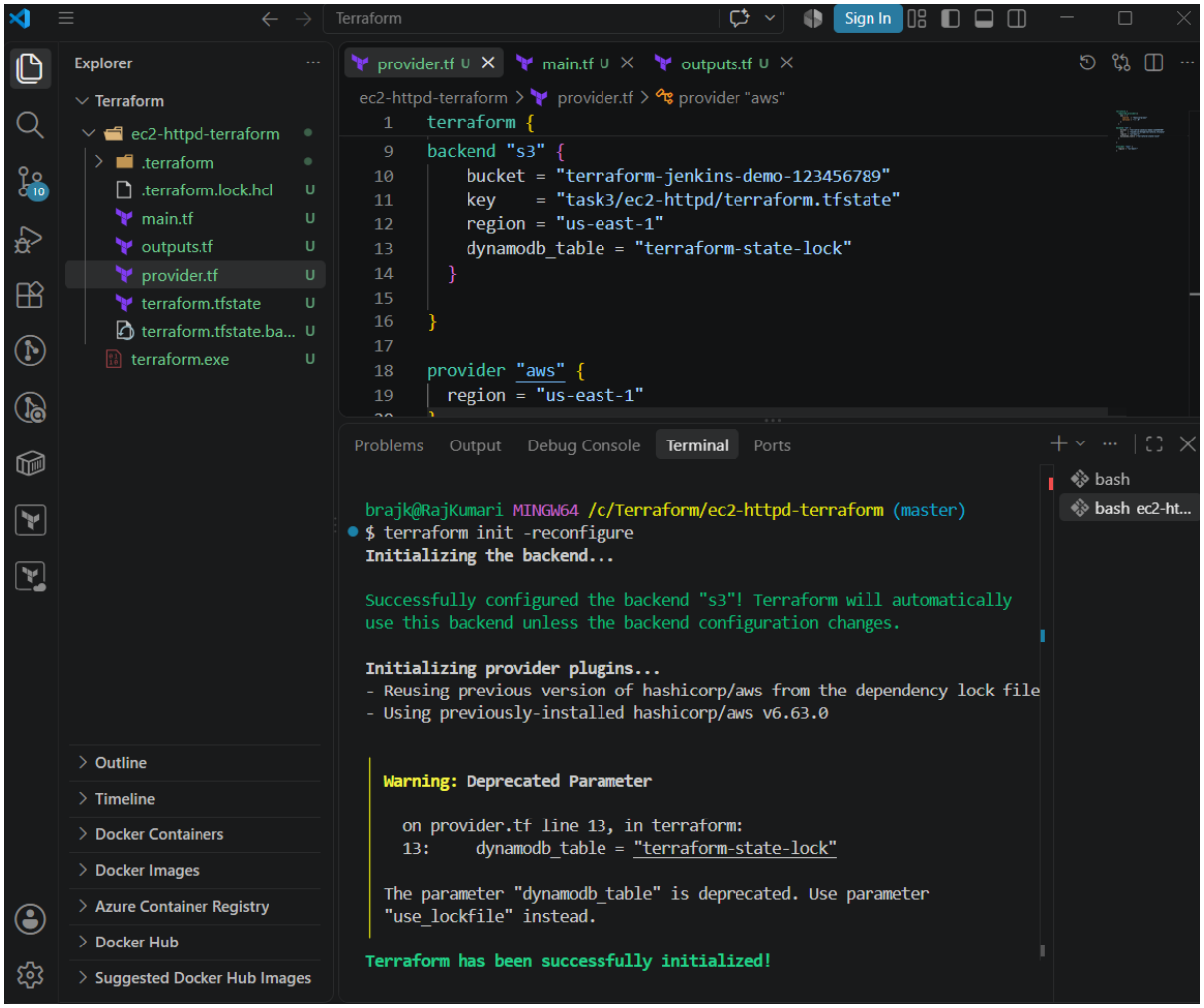
Expected:

```
{
    "Name": "terraform-state-lock",
    "Status": "ACTIVE"
```

Step 3: Configure DynamoDB in provider.tf

You previously configured your S3 backend like this:





Step 5: Verify the backend

Run: terraform plan

Step 6: Understand what happens during terraform apply

When you run:

terraform apply

Terraform acquires a lock in DynamoDB before modifying the state.

Conceptually:

terraform apply

↓

Acquire DynamoDB lock

↓

Read terraform.tfstate from S3

↓

Create/modify AWS resources

↓

Update terraform.tfstate in S3

↓

Release DynamoDB lock

The lock prevents another Terraform process from modifying the same state simultaneously.

S3 → stores Terraform state
DynamoDB → locks the state

The deprecation warning about dynamodb_table is separate from this error. For your assignment specifically asking for **DynamoDB locking**, you can continue with dynamodb_table.